TLP:CLEAR

VULNERABILITY ANALYSIS CVE-2021-44228 · +3

Log4j.

A logging library that executes whatever it is asked to log, in four CVEs across five weeks — three of them created or revealed by the fix for the one before.

ASSESSED SEVERITY — High

SUBJECT TYPE	ATTRIBUTION	FIRST OBSERVED
Vulnerability · remediation cascade	No single actor · state and criminal	December 2021
REPORT DATE	**IS ACTIVE?**	**ALSO KNOWN AS**
19 August 2026	Yes	Log4Shell

EXECUTIVE SUMMARY

Log4Shell is a flaw in Apache Log4j 2, a logging library that is compiled into Java applications and bundled inside appliances rather than bought as a product. Until the fix, anything an application wrote to its log could be made to fetch and execute code from a server the attacker controlled — a username, a header, a chat message. It scored CVSS 10.0 with no credentials and no user interaction required. Within eight weeks of disclosure the same string was being used by Monero miners, commodity ransomware, access brokers selling footholds, and intelligence services of four states, one of which reached a US federal network. Three further CVEs followed in five weeks, each created or revealed by the fix for the one before, ending at version 2.17.1.

> **Only two of the four CVEs are an emergency.** CVE-2021-44228 and CVE-2021-45046 are unauthenticated remote code execution. CVE-2021-45105 is a 5.9 denial of service and CVE-2021-44832 requires the attacker to already control the logging configuration. A patch plan that treats all four as one priority spends its window in the wrong place.

> **The cascade is a chain on your side of the fence, not the attacker's.** Part 2 reads as four linked states because that is the sequence of versions an estate passed through while patching. No attacker needed more than the first one to get in.

> **Your exposure is set by what logs untrusted input, not by an application inventory.** Log4j arrives as a transitive dependency and inside appliances, so the estates that were hit are the ones running a vulnerable copy on something internet-reachable — which an asset list organised by vendor and product will not tell you.

> **Hunting that starts at the disclosure date starts late.** Talos recorded exploitation from around 2 December 2021, a week before public disclosure, and advises backdating compromise hunts by several weeks.

> **The long tail is a patching gap, not a capability gap.** Both documented state intrusions came through unpatched VMware Horizon months after fixed versions shipped, and CISA was still publishing advisories about it in June 2022. Nothing about the attackers improved; the estates simply stayed unpatched.

PART 1

Written analysis

Who the subject is, what it has done in order, and which sectors sit in its path. Part 2 is how a compromise actually unfolds.

OVERVIEW

A library nobody chose, on a path nobody mapped

Apache Log4j 2 is a logging library, not a product. It is compiled into Java applications, bundled inside appliances and shipped as a transitive dependency of software whose authors never chose it, which is why the question after December 2021 was rarely whether an organisation was affected and almost always *where the library was*. CVE-2021-44228, named Log4Shell, carries **CVSS 10.0** with a changed scope: JNDI lookups were resolved inside log messages, so an attacker who could get a string into a log — a username, a User-Agent header, a chat message — could make the logging call fetch and execute code from an LDAP server they controlled. [1] No credentials, no user interaction, and no need to reach the vulnerable component directly. Cisco Talos observed exploit strings firing as they transited internal logging and SIEM infrastructure rather than only on the systems they were aimed at. [5]

What makes these four worth reading as one story is that they are not four independent bugs found by four researchers. **Three were created or revealed by the fix for the one before.** The 2.15.0 release that answered Log4Shell proved incomplete in non-default configurations, which became CVE-2021-45046. [2] The 2.16.0 release that closed that left uncontrolled recursion in self-referential lookups, which became CVE-2021-45105. [3] The 2.17.0 release that closed *that* left a JDBC Appender path to remote code execution, which became CVE-2021-44832. [4] The cascade ends at 2.17.1. This is a chain on the defender's side of the fence: it describes the sequence of versions an organisation passed through while patching, not a sequence of steps an attacker composes. The severities fall away sharply after the first two — CVE-2021-45105 is a 5.9 denial of service and CVE-2021-44832 is a 6.6 that requires the attacker to already control the logging configuration — and only CVE-2021-44228 and CVE-2021-45046 carry the unauthenticated remote code execution that made this event what it was.

The exploitation that followed was unusually broad rather than unusually sophisticated. Mandiant recorded mass scanning from the day of disclosure and described a very long tail, on the reasoning that Log4j sits inside an unknown number of applications and cannot be inventoried by asking. [7] Financially motivated cryptomining crews were first at scale; state-sponsored groups tracked to China, Iran, North Korea and Turkey followed within days; and access brokers used the flaw to build footholds they then sold on to ransomware affiliates. [6] CVE-2021-44228 and CVE-2021-45046 are listed in the CISA Known Exploited Vulnerabilities catalog. [13]

Exploitation, oldest first

2021-12-02

Exploitation predates public disclosure

Cisco Talos records attacker activity against the flaw beginning 2 December 2021, with unverified reports of activity from 1 December — a week before the 9 December public disclosure. Talos advises defenders to backdate compromise hunting by several weeks rather than starting from the disclosure date, because the earliest exploitation was already historical by the time anyone knew to look for it. [5]

2021-12-10

Mass opportunistic scanning and exploitation begins

Mandiant observes mass scanning and exploitation attempts across a large variety of customers beginning the day of disclosure. [7] Palo Alto Unit 42 records **125,894,944 hits** on its Log4j remote code execution signature between 10 December 2021 and 2 February 2022, with the largest spike over 12–16 December. Beyond code execution, attackers use the JNDI lookup itself as an exfiltration channel, crafting strings that interpolate environment variables such as `AWS_SECRET_ACCESS_KEY` into an LDAP or DNS callback so that credentials leave the host inside the resolver query. [8]

2021-12-11

State-sponsored groups and access brokers adopt the flaw

Microsoft reports that groups tracked to China, Iran, North Korea and Turkey moved from experimentation during development to in-the-wild payload deployment. **PHOSPHORUS**, an Iranian actor known to deploy ransomware, acquired and modified the exploit and operationalised it. **HAFNIUM** used it against virtualization infrastructure, fingerprinting targets by DNS, which extended their usual targeting. Multiple access broker groups used it to gain initial access to Linux and Windows networks for resale to ransomware affiliates. Observed post-exploitation payloads include Cobalt Strike, Meterpreter, HabitsRAT, Bladabindi, Webtoos, Tsunami backdoors on Linux hosts, the Mirai botnet and coin miners. Khonsari ransomware is delivered from compromised Minecraft servers running a vulnerable Log4j version. [6]

2021-12-13

AQUATIC PANDA exploits VMware Horizon

CrowdStrike OverWatch observes AQUATIC PANDA, a China-based actor whose documented remit covers intelligence collection and industrial espionage, exploiting a vulnerable VMware Horizon instance. Connectivity checks run as DNS lookups for a subdomain of `dns.1433.eu.org`, executed under the Apache Tomcat service. The intrusion uses `JNDI-Injection-Exploit-1.0.jar`, released on a public GitHub project on 13 December. The actor then enumerates privilege, system and domain detail using native operating system binaries, attempts to discover and stop a third-party EDR service, and runs a Base64-encoded PowerShell command to retrieve further tooling. [9]

2021-12-20

Commodity payloads dominate at volume

Sophos records millions of exploit attempts in customer telemetry, peaking on 15 December and staying elevated through late December. The majority originate from IP addresses in Russia and China, with a single Russian address accounting for 11 percent of identifiable traffic and linked to the Kinsing botnet. Payload families exceed twenty and include XMRig and other Linux coin miners, TellYouThePass ransomware, backdoors, DDoS tools and rootkits. [10] Unit 42 separately observes the Kinsing coinminer, a password stealer exfiltrating the contents of `/etc/passwd` and environment variables over HTTP POST and DNS tunnelling, and a Cobalt Strike server standing up on 18 December. [8]

2022-01-04

NightSky ransomware deployed through VMware Horizon

Microsoft reports DEV-0401, a China-based ransomware operator previously associated with the LockFile, AtomSilo and Rook families, exploiting internet-facing VMware Horizon systems to deploy NightSky ransomware. This marks the shift from opportunistic payloads dropped at scanning scale to deliberate use of the flaw against one specific exposed product. [6]

2022-02

Iranian state actors reach a US federal network

Iranian government-sponsored APT actors exploit Log4Shell on an unpatched VMware Horizon server to gain initial access to a federal civilian executive branch network. They install XMRig cryptomining software, move laterally to the domain controller, compromise credentials, and implant Ngrok reverse proxies on several hosts for persistence. CISA and the FBI conduct the incident response engagement between mid-June and mid-July 2022 and publish the findings that November. IRGC-affiliated actors are separately recorded using the same route into a US municipal government network in the same month. [11]

2022-06

Exploitation of unpatched VMware Horizon continues six months on

CISA and United States Coast Guard Cyber Command publish a joint advisory recording that malicious actors are still exploiting Log4Shell in unpatched VMware Horizon and Unified Access Gateway servers, six months after fixed versions shipped. What keeps the vulnerability productive at this point is the gap between an available patch and a patched estate, not any change in attacker capability. [12]

CONCLUSION

Four CVEs, two emergencies

Log4Shell earned its reputation on reach rather than craft. The exploit is a string; the precondition is that something logs it; and the population of things that log attacker-controlled input turned out to be almost everything. That is why the same flaw produced Monero miners, commodity ransomware, four states' intelligence services and a compromised US federal network inside eight weeks, and why the sector table in this report discriminates less than it usually would. The three CVEs that followed are a different kind of problem: they record what it cost to patch under pressure, with two upgrade cycles spent on issues far less severe than the one that started it.

1. **Only two of the four are the emergency.** CVE-2021-44228 and CVE-2021-45046 are unauthenticated remote code execution. CVE-2021-45105 is a 5.9 denial of service, and CVE-2021-44832 needs the attacker to already control the logging configuration. Treating all four as one priority spends the patch window in the wrong place.

2. **The cascade is a defender's chain, not an attacker's.** Nothing here composes into a multi-step exploit. The sequence describes the versions an estate passed through, so a reader should not infer that an attacker needs more than one of these to get in — the first one alone was always enough.

3. **The version number is not the exposure.** What matters is whether a vulnerable Log4j 2 sits on a path that logs attacker-controlled input, which is why appliances and transitive dependencies mattered more than any application inventory an organisation already had.

4. **Compromise hunting that starts at disclosure starts late.** Talos recorded exploitation roughly a week before 9 December 2021. Any retrospective scoped to the disclosure date has a gap at the front of it.

5. **The long tail is a patching-gap problem, not a capability problem.** Six months after fixed versions shipped, CISA was still publishing advisories about unpatched VMware Horizon. The vulnerability stayed productive because estates stayed unpatched.

RISK & IMPACT

Exposure follows the runtime, not the industry

Sector is a weaker discriminator here than for almost any other subject, because the flaw is in a library rather than in a product one industry happens to buy. The levels below reflect where exploitation was actually observed and reported, not where it was possible.

Government and public administration	CRITICAL	The only Log4Shell intrusions attributed to state actors in the cited reporting are against government — a US federal civilian executive branch network and a US municipal government, both entered through an unpatched VMware Horizon server.
Technology and cloud services	CRITICAL	The products actually exploited — VMware Horizon, Apache Tomcat, Elasticsearch — are this sector's own operating stack, and it carries the largest internet-facing Java estate, which is what the scanning telemetry was finding.
Manufacturing and industrial	HIGH	AQUATIC PANDA, whose documented mission includes industrial espionage, was among the first actors observed using Log4Shell, and used it against VMware Horizon rather than at scanning scale.
Financial services	HIGH	Fits the observed profile closely — large internet-facing Java estates and heavy appliance use — but the cited reporting names no financial victim, so this rests on exposure rather than on observed targeting.
Education and research	MODERATE	Plausible on the same exposure reasoning, and the Minecraft server compromises that delivered Khonsari sit adjacent to it, but no education victim appears in the cited reporting.
Retail and hospitality	LOW	No exploitation against retail appears in any of the cited sources. The level reflects that absence of reporting, not structural protection — a retail estate running a vulnerable Log4j 2 is exactly as exploitable as any other.

Estates with no Java runtime in scope		MINIMAL	Log4j 2 is a Java library, so an estate that runs none is not exposed regardless of industry. This row exists to make the point the rest of the table cannot: for this subject the discriminator is the runtime, not the sector.

Critical — actively targeted
High — fits the victim profile
Moderate — plausible, not observed
Low — outside the pattern
Minimal — no indication

Ask which systems log untrusted input, not which department owns them. Every sector row above is a summary of where exploitation was reported, and for this subject that is a weaker guide than usual. The estates that were hit are the ones that ran a vulnerable Log4j 2 on something reachable from the internet, and the two that made the advisories ran it inside VMware Horizon.

For *how* a compromise actually unfolds — the techniques, the tooling and the order they run in — see the companion attack-flow report.

PART 1 SOURCES

What this analysis is built on

1. NVD — CVE-2021-44228 (Log4Shell, CVSS 10.0, CWE-917) — https://nvd.nist.gov/vuln/detail/CVE-2021-44228
2. NVD — CVE-2021-45046 (incomplete fix, CVSS 9.0, CWE-917) — https://nvd.nist.gov/vuln/detail/CVE-2021-45046
3. NVD — CVE-2021-45105 (uncontrolled recursion, CVSS 5.9, CWE-674) — https://nvd.nist.gov/vuln/detail/CVE-2021-45105
4. NVD — CVE-2021-44832 (JDBC Appender RCE, CVSS 6.6, CWE-74) — https://nvd.nist.gov/vuln/detail/CVE-2021-44832
5. Cisco Talos — Threat Advisory: critical Apache Log4j vulnerability being exploited in the wild — https://blog.talosintelligence.com/apache-log4j-rce-vulnerability/
6. Microsoft Threat Intelligence — Guidance for preventing, detecting and hunting for exploitation of the Log4j 2 vulnerability — https://www.microsoft.com/en-us/security/blog/2021/12/11/guidance-for-preventing-detecting-and-hunting-for-cve-2021-44228-log4j-2-exploitation/
7. Mandiant (Google Cloud) — Log4Shell initial exploitation and mitigation recommendations — https://cloud.google.com/blog/topics/threat-intelligence/log4shell-recommendations
8. Palo Alto Unit 42 — Another Apache Log4j vulnerability is actively exploited in the wild (CVE-2021-44228) — https://unit42.paloaltonetworks.com/apache-log4j-vulnerability-cve-2021-44228/
9. CrowdStrike — OverWatch exposes AQUATIC PANDA in possession of Log4Shell exploit tools — https://www.crowdstrike.com/en-us/blog/overwatch-exposes-aquatic-panda-in-possession-of-log-4-shell-exploit-tools/
10. Sophos X-Ops — Logjam: Log4j exploit attempts continue in globally distributed scans and attacks — https://www.sophos.com/en-us/blog/logjam-log4j-exploit-attempts-continue-in-globally-distributed-scans-attacks
11. CISA and FBI — AA22-320A, Iranian government-sponsored APT actors compromise federal network, deploy crypto miner and credential harvester — https://www.cisa.gov/news-events/cybersecurity-advisories/aa22-320a
12. CISA and USCG Cyber Command — AA22-174A, malicious cyber actors continue to exploit Log4Shell in VMware Horizon systems — https://www.cisa.gov/news-events/cybersecurity-advisories/aa22-174a
13. CISA — Known Exploited Vulnerabilities Catalog — https://www.cisa.gov/known-exploited-vulnerabilities-catalog
14. Tenable — Frequently asked questions about Log4Shell and associated vulnerabilities — https://www.tenable.com/blog/cve-2021-44228-cve-2021-45046-cve-2021-4104-frequently-asked-questions-about-log4shell

Confidence and sourcing. The CVE identifiers, scores, affected ranges and fixed versions are reference data from NVD. The exploitation account is drawn from primary vendor and government reporting, and where a claim rests on one vendor only — the AQUATIC PANDA intrusion, the DEV-0401 NightSky deployment — the report names that vendor rather than presenting it as consensus. The actor attributions here are *reported*, not assessed by this analysis. **The *Is active?* field needs a caveat:** the most recent documented intrusion in these sources is from 2022, and the *Yes* rests on continued listing in the CISA KEV catalog and on Mandiant's long-tail assessment rather than on a fresh incident report. Sector risk levels are analyst assessments, not measured values; they deliberately share a colour ramp with CVSS severity but are rendered as meters rather than scores so the two are not confused.

PART 2

ATT&CK attack flow

How a compromise unfolds, tactic by tactic and in order. Part 1 established who is at risk and why; this part is how they get hit.

VULNERABILITY REPORT **APACHE LOG4J 2 · DEC 2021 – JAN 2022**

Log4j.

Four CVEs in five weeks, and three of them were **created or revealed by the fix for the one before**. This is a chain — but a chain on the defender's side of the fence. Each upgrade closed one flaw and moved you to a version exposed to the next.

CASCADE `CVE-2021-44228` `CVE-2021-45046` `CVE-2021-45105` `CVE-2021-44832`

VULNERABILITIES	HIGHEST CVSS	IN CISA KEV
4	10.0	2 of 4
PRODUCT	**EXPOSURE**	**CASCADE ENDS**
Apache Log4j 2	Any logged input	2.17.1

VULNERABILITIES

Every fix moved you to the next flaw

Read this as a cascade of *deployed versions*, not of attacker capability. The states below are what you were running; each link is the CVE that version was exposed to, and the fix for it is what carried you to the state beneath.

STEP	CVE	CVSS	EXPLOITATION	WHAT IT DOES	CLOSED BY
1	CVE-2021-44228	10.0	KEV	Unauthenticated RCE from any logged string	2.15.0
2	CVE-2021-45046	9.0	KEV	Code execution — the 2.15.0 fix was incomplete	2.16.0
3	CVE-2021-45105	5.9	NO KNOWN EXPLOITATION	Denial of service via uncontrolled recursion	2.17.0
4	CVE-2021-44832	6.6	NO KNOWN EXPLOITATION	RCE via JDBC Appender, needs config control	2.17.1

YOU ARE RUNNING

Log4j 2.14.1 or earlier — the state most of the world was in on 9 December 2021

CVE-2021-44228 · 10.0 CRITICAL · KEV · CWE-917 Expression Language Injection

Log4Shell. JNDI lookups are evaluated inside log messages, so any string an attacker can get logged — a header, a username, a filename — becomes an instruction to fetch and run remote Java. No authentication, no user interaction.

> Alone: **this is the only one of the four that needed nothing but a logged string.** It is why the other three exist — the rush to patch it produced them.

YOU UPGRADE TO

Log4j 2.15.0 — believed to be "the Log4Shell fix"

CVE-2021-45046 · 9.0 CRITICAL · KEV · CWE-917 Expression Language Injection

The incomplete fix. NVD states it plainly: the patch for CVE-2021-44228 in 2.15.0 "was incomplete in certain non-default configurations". Attacker-controlled Thread Context Map data reaching a Pattern Layout with a Context Lookup still hits the same JNDI primitive.

> Alone: **only reachable on 2.15.0 and non-default layouts** — which means the organisations exposed to it were, by definition, the ones that patched fastest.

YOU UPGRADE TO

Log4j 2.16.0 — message lookups removed entirely

`CVE-2021-45105` `5.9 MEDIUM` `NO KNOWN EXPLOITATION` `CWE-674 Uncontrolled Recursion`

Crash, not compromise. Self-referential lookups were still not guarded, so a crafted string drives infinite recursion and terminates the thread.

> Alone: **availability only — no code execution and no data access.** The vector says so outright: `C:N/I:N/A:H`. This is the one step in the cascade that cannot lead to compromise.

YOU UPGRADE TO

Log4j 2.17.0 — recursion guarded

`CVE-2021-44832` `6.6 MEDIUM` `NO KNOWN EXPLOITATION` `CWE-74 Injection`

The long tail. A JDBC Appender pointed at a hostile JNDI LDAP data source still executes code — but only for an attacker who can already rewrite your logging configuration.

> Alone: **requires privileges an external attacker does not have** — `PR:H` in the vector. An adversary who can already edit config files on your server has better options than this one.

CASCADE ENDS

Log4j 2.17.1 — or 2.12.4 for Java 7, 2.3.2 for Java 6

This is a remediation cascade, not an exploit chain. An attacker never composes these — they exploit exactly one, whichever matches the version they find. The dependency runs the other way: each flaw was surfaced by the fix for the previous one, which is why CVE-2021-45046's own record describes 2.15.0's patch as incomplete.

The practical consequence is that **partial progress through this cascade was not safety**. Teams that upgraded to 2.15.0 in the first 72 hours and stopped were still exposed. The only state that exits the cascade is 2.17.1.

And the ordering matters far less than it looks, because one action resolves all four. The real difficulty in December 2021 was never sequencing — it was **discovery**: `log4j-core` is a transitive dependency, repackaged inside shaded JARs and embedded in vendor appliances, and most inventories simply could not answer where it was.

THE INTRUSION

One attack path mattered

Of the four, only CVE-2021-44228 produced mass exploitation at scale, so it gets the flow. The others are deliberately not drawn — see the note beneath.

CVE-2021-44228 — ATTACK PATH

01

INITIAL ACCESS

Send a string that will be written to a log

`T1190` Exploit Public-Facing Application — the payload is ordinary request data, so the entry point is wherever the application logs untrusted input. `via CVE-2021-44228`

↓ *the string is not the payload — it is an instruction to go and collect one*

02

COMMAND AND CONTROL

The victim fetches its own payload

Ordered before Execution because that is the real sequence: the JNDI lookup makes an *outbound* connection to attacker infrastructure and retrieves a Java class, which only then runs. That inversion is why egress filtering worked as an emergency control for organisations who could not patch immediately.

`T1105` Ingress Tool Transfer — the server resolves the attacker-controlled `ldap://` endpoint and downloads a remote class file.

↓ *the retrieved class is loaded and run inside the running JVM*

03

EXECUTION

Arbitrary Java, with the application's privileges

`T1059` Command and Scripting Interpreter — code runs as the service account the application uses, which on many deployments is more privileged than it needs to be.

↓ *at internet scale, what ran next was overwhelmingly commodity*

04

IMPACT — OPERATOR-DEPENDENT

Mostly coin miners, sometimes far worse

The vulnerability decided none of this. Opportunistic mass exploitation skewed heavily toward cryptomining and botnet recruitment because those monetise without human effort; targeted operators used identical access for espionage and ransomware staging.

`T1496.001` Resource Hijacking: Compute Hijacking — commodity miners deployed at scale against exposed hosts.

Why the other three have no flow. Drawing one for each would imply four comparable intrusion paths, and there were not four.

CVE-2021-45046 reaches the same JNDI primitive by a narrower route — its flow would duplicate the one above with a higher attack complexity and a smaller population. **CVE-2021-45105** terminates at denial of service; there is no execution phase to draw, and its two-phase flow would say only "send string, thread dies". **CVE-2021-44832** begins from an attacker who already controls your logging configuration, so it has no Initial Access phase at all.

Tactic icon and label warm as exploitation advances — The spine tracks the same progression

`T####` ATT&CK technique ID — `via CVE` The CVE that enables this step — `10.0` `5.9` CVSS v3.1 base score

`KEV` In CISA's Known Exploited Vulnerabilities catalog

REMEDIATION

One upgrade exits the whole cascade

There is no sequencing decision here. Every step above is closed by a single target version, which is why effort belongs in finding the library rather than in ordering the fixes.

Go straight to 2.17.1

Or `2.12.4` on Java 7, `2.3.2` on Java 6. Do not stop at 2.15.0 or 2.16.0 — those are intermediate states inside the cascade, not exits from it.

PATCH

Find it before you fix it

Audit resolved dependencies, not manifests. `log4j-core` arrives transitively and hides inside shaded and repackaged JARs that a top-level dependency list will not show.

DISCOVERY

Treat stopgaps as stopgaps

Removing `JndiLookup.class` and setting `formatMsgNoLookups` circulated widely as mitigations. Both were bridging measures for the first flaw only and are superseded by upgrading.

MITIGATION

CVE	CVSS	CWE	AFFECTED UP TO	FIXED IN
CVE-2021-44228	10.0	CWE-917	2.15.0	2.15.0 / 2.12.2
CVE-2021-45046	9.0	CWE-917	2.15.x	2.16.0 / 2.12.2
CVE-2021-45105	5.9	CWE-674	2.16.0	2.17.0 / 2.12.3 / 2.3.1
CVE-2021-44832	6.6	CWE-74	2.17.0	2.17.1 / 2.12.4 / 2.3.2

PART 2 SOURCES

What this mapping is built on

1 NVD — CVE-2021-44228 (Log4Shell) — https://nvd.nist.gov/vuln/detail/CVE-2021-44228

2 NVD — CVE-2021-45046 — https://nvd.nist.gov/vuln/detail/CVE-2021-45046

3 NVD — CVE-2021-45105 — https://nvd.nist.gov/vuln/detail/CVE-2021-45105

4 NVD — CVE-2021-44832 — https://nvd.nist.gov/vuln/detail/CVE-2021-44832

CVSS scores, vectors, CWE assignments and affected and fixed versions verified against NVD.

Read the cascade correctly. The links are ordered by *remediation*, not by exploitation. An attacker exploits one of these four — whichever matches the version in front of them — and never composes them.

Derived mapping — read before citing. MITRE ATT&CK publishes no Log4j mapping, so the techniques on the CVE-2021-44228 flow were **derived by analysis**. Every ID and tactic was verified to exist against attack.mitre.org — including `T1496.001`, now a sub-technique of Resource Hijacking rather than the flat technique older references cite — but applying them here is analytical judgment. CVE facts are NVD data and are not derived.